CESI
ÉCOLE D'INGÉNIEURS

Stockage & Big Data

Corbeille d'exercices | MongoDB

Mercredi 28 octobre 2026



Mazou Marine
Fisa Info 25–28
CESI

TABLE DES MATIÈRES

Introduction **2**

Étude des données **3**

Partie 1 : installation de MongoDB **4**

Partie 2 : import des données **5**

Partie 3 : requêtes **6**
1. Découverte de la base (Q1 à Q3) 6
2. Collection netflix (Q4 à Q8 et Q12) 7
3. Collection restaurants (Q9 à Q11) 9
4. Collection height (Q13 à Q16) 10

Synthèse et écarts à l'énoncé **12**
1. Réponses 12
2. Écarts et choix d'interprétation 12

Conclusion **13**

INTRODUCTION

MongoDB est une base de données **NoSQL orientée documents**. Elle ne stocke pas de lignes dans des tables, mais des **documents** au format JSON (stockés en BSON), regroupés en **collections**, elles-mêmes rangées dans une **base**. Un document est un ensemble de paires champ : valeur. Une valeur peut être un nombre, un texte, une date, un tableau ou un autre document : les données se décrivent donc par imbrication, sans jointure. Deux documents d'une même collection n'ont pas obligatoirement les mêmes champs : le schéma n'est pas imposé par la base.

Le tableau compare MongoDB au modèle relationnel et à Hive, vu dans le workshop précédent :

Notion	Relationnel (SQL)	Hive	MongoDB
Ensemble	Base	Base	Base
Regroupement	Table	Table (fichiers HDFS)	Collection
Unité de donnée	Ligne	Ligne d'un fichier	Document (JSON)
Schéma	Fixé avant l'écriture	Décrit dans le metastore, appliqué à la lecture	Libre, chaque document porte ses champs
Imbrication	Non, jointures	Types complexes limités	Documents et tableaux imbriqués
Langage	SQL	HiveQL	Méthodes JavaScript (`find`, `aggregate`)

La corbeille d'exercices comporte trois étapes : installer MongoDB, importer trois jeux de données dans la base `corbeille`, puis répondre à seize questions avec le shell **mongosh**. Le compte-rendu commence par une étude des données, car leur forme décide des requêtes à écrire. Les valeurs données viennent de l'exécution réelle du 28 octobre 2026 ; les réponses chiffrées (Q2 et Q4 à Q16) ont de plus été recalculées en Python, avec le même résultat.

ÉTUDE DES DONNÉES

Trois fichiers sont fournis. Chacun devient une collection de la base `corbeille` :

Fichier	Format	Documents	Champs
`restaurants.json`	JSON	25 357	`name`, `restaurant_id`, `borough`, `cuisine`, `address` (`building`, `coord`, `street`, `zipcode`), `grades`
`netflix_dataset.csv`	CSV	8 807	`show_id`, `type`, `title`, `director`, `cast`, `country`, `date_added`, `release_year`, `rating`, `duration`, `listed_in`, `description`
`heightbycountry_dataset.csv`	CSV	199	`Rank`, `Country Name`, `Male Height in Cm`, `Female Height in Cm`, `Male Height in Ft`, `Female Height in Ft`

Le fichier `restaurants.json` contient un document par ligne. Ses documents sont imbriqués : l'adresse est un sous-document, qui contient lui-même les coordonnées, et les notes forment un tableau de sous-documents. La figure montre cette structure, relevée sur le premier document de la collection.

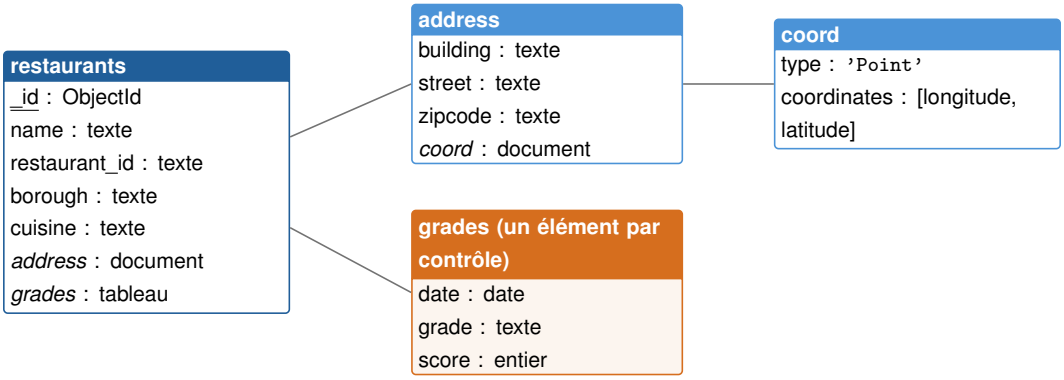


Figure 1 – Structure d'un document de la collection `restaurants` : `address` et `coord` sont imbriqués, `grades` est un tableau (4 éléments pour le premier document).

Les données ont des particularités qui influencent les requêtes :

Particularité	Constat	Conséquence
`country` **multi-valeurs**	Un film a un ou plusieurs pays séparés par des virgules (par exemple « France, Belgium »)	Filtre par expression régulière en Q7 et Q8, découpage en Q12
`duration` **en texte**	Valeur du type `'90 min'` ou `'2 Seasons'`, jamais un nombre	Comparaison avec la chaîne `'90 min'`
Champs vides	Les cases vides du CSV (`director`, `cast`, `country`...) sont importées comme des chaînes vides `''`, et non comme des champs absents	Les vides sont à exclure en Q12
`borough` « **Missing** »	Valeur présente dans la collection, qui n'est pas un quartier	Q11 : 6 valeurs, dont 5 vrais quartiers
Cuisine `'American '`	Une valeur avec une espace finale, distincte de `'American'`	Comptée telle quelle en Q9
Cuisine mal encodée	`'CafÃ©/Coffee/Tea'` existe à côté de `'Café/Coffee/Tea'`, déjà dans le fichier source	Deux valeurs distinctes en Q9
Noms de champs avec espaces	`Male Height in Cm`, `Country Name`...	Champ entre guillemets, `'$Male Height in Cm'` en agrégation

PARTIE 1 : INSTALLATION DE MONGODB

L'énoncé demande la dernière version de MongoDB Community Server (alors la 5.0.6), la suite *Database Tools*, puis l'ajout des dossiers `bin` au `PATH`. Voici ce qui a été installé et lancé sur le poste Windows :

Élément	Version utilisée	Remarque
Serveur MongoDB Community	8.3	Service Windows, écoute sur `localhost:27017`
`mongosh`	2.14.0	Shell des seize requêtes
MongoDB Database Tools	100.20.0	Archive zip, décompressée dans `C:\Users\mazou\mongodb-tools`

Les écarts à l'énoncé sont les suivants :

- **Version** : le serveur est en 8.3 et non en 5.0.6, puisque la 5.0.6 n'est plus la dernière version. Les requêtes de la corbeille n'utilisent que des fonctions anciennes (`find`, `distinct`, `aggregate`), mais elles n'ont pas été rejouées sur une 5.0.
- **Database Tools** : installés depuis l'archive zip et non depuis un installateur. Le dossier n'est pas dans le `PATH` : le script d'import appelle `mongoimport.exe` par son chemin complet.

PARTIE 2 : IMPORT DES DONNÉES

L'import est écrit dans un script PowerShell, `import.ps1`, qui peut être rejoué grâce à l'option `--drop` (elle supprime la collection avant de la recréer). Les deux fichiers CSV utilisent `--type csv --headerline`, qui prend la première ligne comme noms de champs. Le fichier JSON, qui contient un document par ligne, s'importe sans option de format :

```
mongoimport --uri mongodb://localhost:27017 --db corbeille --collection netflix
↪  --drop --type csv --headerline --file netflix_dataset.csv
mongoimport --uri mongodb://localhost:27017 --db corbeille --collection height
↪  --drop --type csv --headerline --file heightbycountry_dataset.csv
mongoimport --uri mongodb://localhost:27017 --db corbeille --collection
↪  restaurants --drop --file restaurants.json
```

L'énoncé ne cite que `netflix` et `restaurants`, mais les questions 13 à 16 portent sur les tailles : la collection `height` est donc importée aussi. Le résultat réel est le suivant, sans aucun échec :

Collection	Documents importés	Documents en échec
`netflix`	8 807	0
`height`	199	0
`restaurants`	25 357	0

Ces nombres sont ceux annoncés par l'énoncé (25 357, 8 807 et 199). Avec un fichier CSV, `mongoimport` devine le type de chaque valeur. La capture montre l'exécution réelle de `import.ps1` dans une console PowerShell ; elle a été refaite le 28 octobre 2026 à 16 h 55 et donne les mêmes nombres. Deux contrôles confirment que le résultat est correct : `release_year` est un entier pour les 8 807 documents (requête de vérification sur `$type`), et les tailles sont numériques, puisque leur moyenne se calcule (Q13).

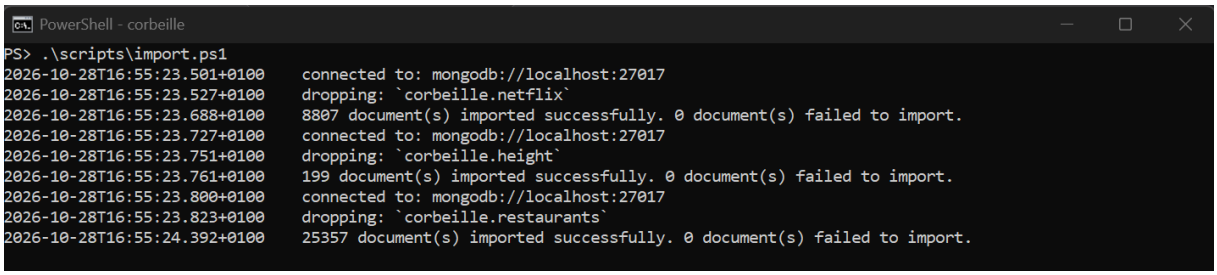


Figure 2 – Exécution de `import.ps1` (console PowerShell) : trois collections importées, 0 échec.

PARTIE 3 : REQUÊTES

Les seize requêtes sont écrites dans `requetes.js`, lancé par `mongosh corbeille --quiet --file requetes.js`. Pour les types « films » et « séries TV », la collection `netflix` utilise le champ `type`, qui vaut `'Movie'` ou `'TV Show'`.

1. DÉCOUVERTE DE LA BASE (Q1 À Q3)

Q1. Liste des collections de la base.

```
db.getCollectionNames()
```

Résultat : `['restaurants', 'netflix', 'height']`. La base contient bien les trois collections importées (l'ordre de la liste n'est pas garanti : après le rejeu de l'import de la capture, les collections sont renvoyées dans l'ordre `restaurants`, `height`, `netflix`).

Q2. Nombre de documents des collections `netflix` et `restaurants`.

```
db.netflix.countDocuments({})
db.restaurants.countDocuments({})
```

Résultat : **8 807** documents dans `netflix` et **25 357** dans `restaurants`, soit les nombres de l'énoncé. La capture montre ces deux premières requêtes dans mongosh, avec la liste des collections.

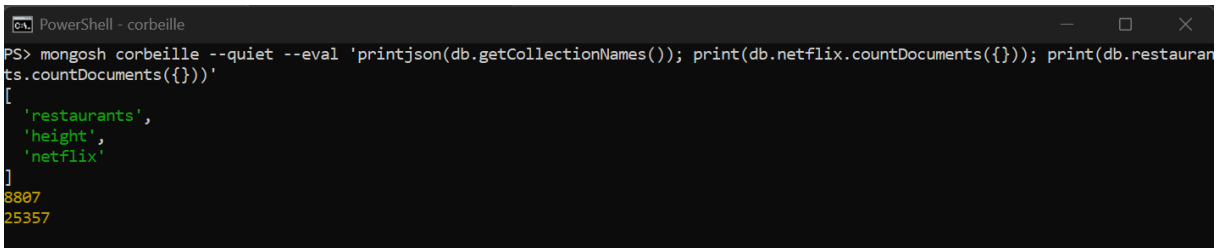


Figure 3 – Q1 et Q2 dans la console : liste des collections, puis `countDocuments` sur `netflix` et `restaurants`.

Q3. Un document de chaque collection et la liste de leurs champs.

```
const docNetflix = db.netflix.findOne();
print("Champs netflix : " + Object.keys(docNetflix).join(", "));
const docResto = db.restaurants.findOne();
print("Champs restaurants : " + Object.keys(docResto).join(", "));
```

Les champs relevés sont les suivants. Le champ `_id` est ajouté par MongoDB à chaque document :

Collection	Champs du document
netflix	`_id, show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in, description`
restaurants	`_id, address, borough, cuisine, grades, name, restaurant_id`

Le document `netflix` obtenu est la série `s2` (champ `cast` abrégé) :

Champ	Valeur
`show_id`	s2
`type`	TV Show
`title`	Blood & Water
`director`	(chaîne vide)
`cast`	Ama Qamata, Khosi Ngema, Gail Mabalane, . . . (19 noms)
`country`	South Africa
`date_added`	September 24, 2021
`release_year`	2021
`rating, duration`	TV-MA, 2 Seasons

Le document `restaurants` obtenu est « Riviera Caterer », à Brooklyn, avec la cuisine `'American '` et quatre notes (de 2011 à 2014). Sa structure est celle de la figure 1. On note que `director` est présent mais vide : c'est la conséquence de l'import CSV décrite plus haut.

2. COLLECTION NETFLIX (Q4 À Q8 ET Q12)

Q4. Nombre de films sortis en 2016.

```
db.netflix.countDocuments({ type: "Movie", release_year: 2016 })
db.netflix.countDocuments({ release_year: 2016 })   // variante
```

Résultat : **658 films**. La question parle de films ; la variante qui ne filtre pas sur le type compte aussi les séries et donne 902 documents sortis en 2016.

Q5 et Q6. Nombre de films et de séries TV.

```
db.netflix.countDocuments({ type: "Movie" })
db.netflix.countDocuments({ type: "TV Show" })
```

Résultat : **6 131 films** et **2 676 séries TV**. Leur somme, 8 807, est le nombre total de documents : il n'y a pas d'autre type. Les films représentent environ 70 % du catalogue (6 131 sur 8 807).

Q7. Nombre de films français dont la durée est de 90 minutes. Le champ `country` peut contenir plusieurs pays : le filtre cherche donc « France » dans le texte avec une expression régulière. Comme `duration` est un texte, on compare avec `'90 min'`.

```
db.netflix.countDocuments({ type: "Movie", country: /France/, duration: "90 min"
↪ })
db.netflix.countDocuments({ type: "Movie", country: "France", duration: "90 min"
↪ })   // variante
```

Résultat : **6 films**. La variante, qui exige que `country` soit exactement `'France'`, donne **0** : les films français de 90 minutes ont tous plusieurs pays de production. Un filtre sur la valeur exacte aurait donc donné une réponse fausse sans avertissement.

Q8. Nombre de films américains sortis depuis 2020. Même principe pour le pays, avec `$gte` sur l'année.

```
db.netflix.countDocuments({ type: "Movie", country: /United States/, release_year:
↪ { $gte: 2020 } })
```

Résultat : **265 films**. « Depuis 2020 » est interprété comme 2020 inclus (`$gte`).

Q12. Nombre de pays pour lesquels la collection contient au moins un film. La question est la plus délicate : un champ `country` contient une liste de pays. Il faut donc découper le texte, mettre un pays par document, nettoyer, puis compter les valeurs distinctes. La figure montre les étapes du pipeline d'agrégation.

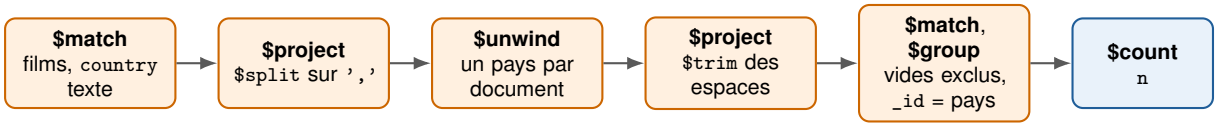


Figure 5 – Pipeline d'agrégation de Q12 : de la liste de pays d'un film au nombre de pays distincts.

```
db.netflix.aggregate([
  { $match: { type: "Movie" } },
  { $match: { country: { $type: "string" } } },
  { $project: { pays: { $split: ["$country", ","] } } },
  { $unwind: "$pays" },
  { $project: { pays: { $trim: { input: "$pays" } } } },
  { $match: { pays: { $ne: "" } } },
  { $group: { _id: "$pays" } },
  { $count: "n" }
])
```

Le `$trim` est nécessaire : après un découpage sur la virgule, `' Belgium'` (avec une espace) et `'Belgium'` seraient comptés comme deux pays. Le second `$match` retire les pays vides, issus des champs `country` vides. Résultat : **117 pays** avec au moins un film. La variante sans filtre sur le type (films et séries) donne 122 pays. La capture montre le fichier d'agrégation et son résultat (dans le fichier, l'agrégation est enveloppée dans `printjson(...toArray())` pour afficher le résultat).

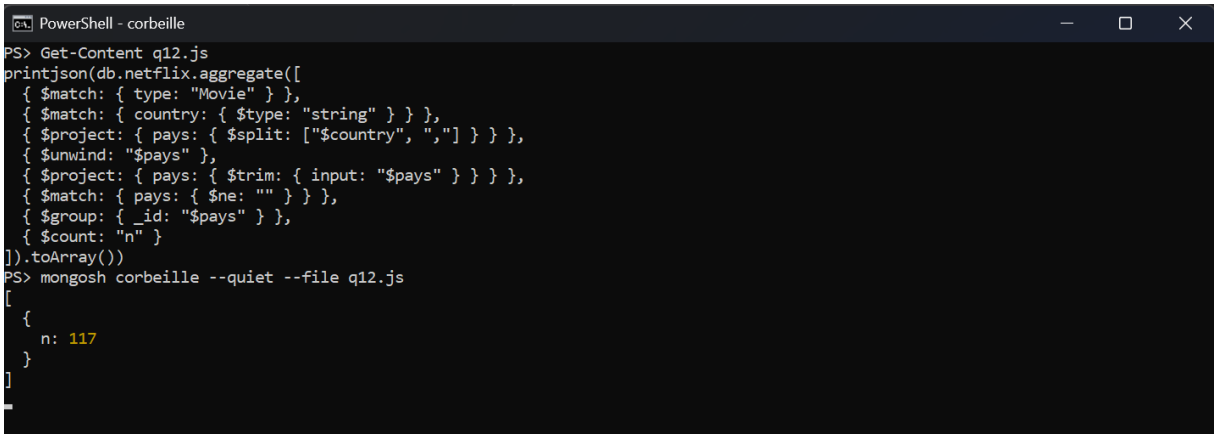


Figure 6 – Q12 dans mongosh : pipeline d'agrégation et résultat `n: 117`.

3. COLLECTION RESTAURANTS (Q9 À Q11)

Q9. Liste des types de restaurants (cuisine). La méthode `distinct` renvoie les valeurs différentes d'un champ, triées.

```
const cuisines = db.restaurants.distinct("cuisine");
print("Nombre de cuisines distinctes : " + cuisines.length);
```

Résultat : **85 valeurs distinctes**. La liste complète est trop longue pour être reproduite ; le tableau donne ses extrémités et les cas particuliers.

Partie de la liste	Valeurs
Début	Afghan, African, `'American '`, Armenian, Asian, Australian, Bagels/Pretzels, Bakery
Cas particuliers	`'American '` (espace finale), `'CafÃ©/Coffee/Tea'` et `'Café/Coffee/Tea'`
Valeurs composées	Chinese/Cuban, Chinese/Japanese, Creole/Cajun, Pizza/Italian, Hotdogs/Pretzels
Valeurs génériques	Not Listed/Not Applicable, Other
Fin	Thai, Turkish, Vegetarian, Vietnamese/Cambodian/Malaysia

Les 85 valeurs sont des valeurs brutes. Si les deux variantes de « Café » étaient regroupées, la liste compterait 84 types. Ces défauts sont signalés mais non corrigés : la question demande la liste telle qu'elle est stockée.

Q10. Nombre de restaurants dans le Queens.

```
db.restaurants.countDocuments({ borough: "Queens" })
```

Résultat : **5 656 restaurants**, soit environ 22 % des 25 357 restaurants de la collection. La capture montre l'exécution de Q7 (résultat 6) et de Q10 depuis un fichier, lancé avec `mongosh --file`.



Figure 4 – Q7 (6 films français de 90 min) et Q10 (5 656 restaurants dans le Queens), exécutés dans mongosh.

Q11. Liste des quartiers qui contiennent au moins un restaurant.

```
db.restaurants.distinct("borough")
```

Résultat : **6 valeurs**, `'Bronx'`, `'Brooklyn'`, `'Manhattan'`, `'Missing'`, `'Queens'` et `'Staten Island'`. `'Missing'` n'est pas un quartier : c'est la valeur des restaurants dont le quartier est inconnu. New York compte cinq quartiers, ce qui correspond aux cinq autres valeurs.

4. COLLECTION HEIGHT (Q13 À Q16)

Q13 et Q14. Taille moyenne des hommes et des femmes, tous pays confondus. L'opérateur `$avg` calcule la moyenne dans un `$group` sans clé (`_id: null`). Les noms de champs contiennent des espaces : ils sont écrits entre guillemets, précédés de `$`.

```
db.height.aggregate([{ $group: { _id: null, m: { $avg: "$Male Height in Cm" } }
↪ }])
db.height.aggregate([{ $group: { _id: null, m: { $avg: "$Female Height in Cm" } }
↪ }])
```

Résultat : **173,09 cm** pour les hommes (valeur brute 173,08904522613065) et **160,94 cm** pour les femmes (160,9429145728643). Il s'agit d'une moyenne simple des 199 pays, non pondérée par la population : ce n'est pas la taille moyenne des habitants du monde. Les hommes sont en moyenne plus grands de 12,15 cm.

Q15 et Q16. Pays aux hommes les plus petits et aux femmes les plus grandes. Un tri suivi de `limit(3)` donne le premier pays et les deux suivants, pour vérifier l'absence d'ex æquo.

```
db.height.find({}, { _id: 0, "Country Name": 1, "Male Height in Cm": 1 })
  .sort({ "Male Height in Cm": 1 }).limit(3)
db.height.find({}, { _id: 0, "Country Name": 1, "Female Height in Cm": 1 })
  .sort({ "Female Height in Cm": -1 }).limit(3)
```

Rang	Hommes, plus petits	Taille (cm)	Rang	Femmes, plus grandes	Taille (cm)
1	Timor-Leste	160,13	**1**	Netherlands	170,36
2	Laos	162,78	**2**	Montenegro	169,96
3	Solomon Islands	163,07	**3**	Denmark	169,47

Les réponses sont **Timor-Leste, 160,13 cm** (Q15) et **Netherlands, 170,36 cm** (Q16). Il n'y a pas d'ex æquo : les tailles du deuxième pays sont différentes. Les Pays-Bas sont aussi au rang 1 du fichier source pour les hommes (183,78 cm). La capture montre l'exécution de Q15 et Q16 (script de sortie compact, trois pays par tri), puis la dernière figure résume les résultats de Q5, Q6 et de Q13 à Q16.

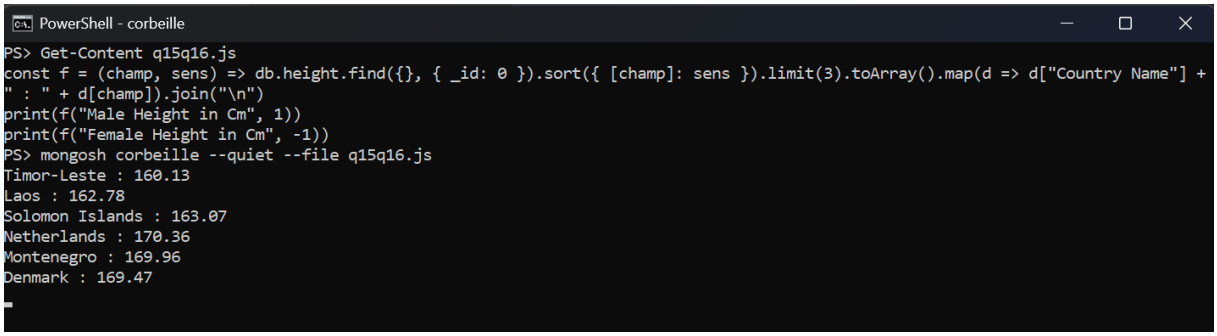


Figure 7 – Q15 et Q16 dans mongosh : les trois pays aux hommes les plus petits, puis aux femmes les plus grandes.

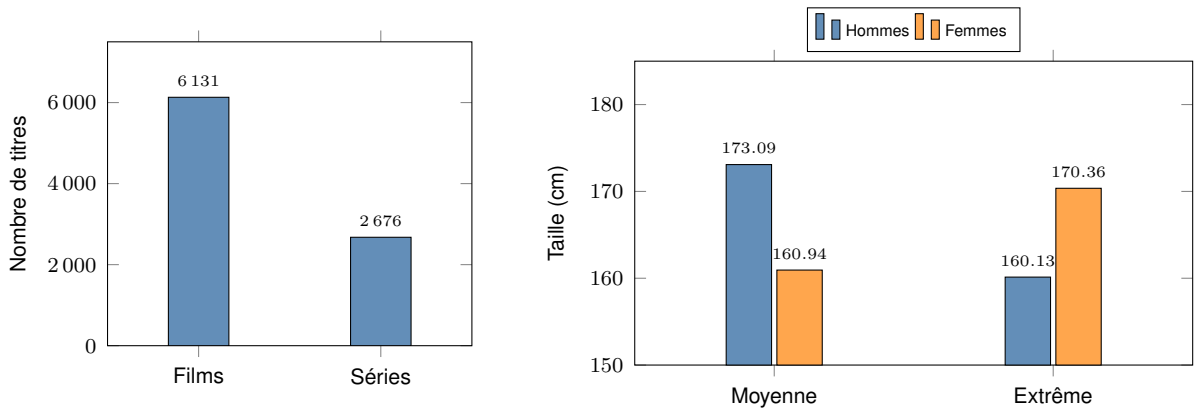


Figure 8 – À gauche : films et séries de la collection `netflix` (Q5, Q6). À droite : tailles en cm, moyenne des 199 pays (Q13, Q14) et valeur extrême (Q15 : hommes les plus petits, Timor-Leste ; Q16 : femmes les plus grandes, Netherlands). L'axe commence à 150 cm.

SYNTHÈSE ET ÉCARTS À L'ÉNONCÉ

1. RÉPONSES

Le tableau regroupe les réponses aux seize questions, telles que renvoyées par `mongosh` :

Q	Question	Réponse
1	Collections de la base	restaurants, netflix, height
2	Documents de netflix et de restaurants	8 807 et 25 357
3	Champs d'un document	13 champs (netflix), 7 (restaurants)
4	Films sortis en 2016	658
5	Films	6 131
6	Séries TV	2 676
7	Films français de 90 min	6
8	Films américains depuis 2020	265
9	Types de cuisine	85 valeurs distinctes
10	Restaurants dans le Queens	5 656
11	Quartiers	6 valeurs (dont « Missing »)
12	Pays avec au moins un film	117
13	Taille moyenne des hommes	173,09 cm
14	Taille moyenne des femmes	160,94 cm
15	Hommes les plus petits	Timor-Leste, 160,13 cm
16	Femmes les plus grandes	Netherlands, 170,36 cm

2. ÉCARTS ET CHOIX D'INTERPRÉTATION

- **Version et installation** : serveur 8.3, `mongosh` 2.14.0 et Database Tools 100.20.0 (archive zip, chemin complet de `mongoimport`, `PATH` inchangé), au lieu de la 5.0.6 de l'énoncé.
- **Collection `height`** : importée en plus de `netflix` et `restaurants`, car Q13 à Q16 en ont besoin.
- **« Films » et « séries »** : interprétés comme `type = 'Movie'` et `type = 'TV Show'`. Les variantes sans filtre de type sont données pour Q4 (902) et Q12 (122).
- **Q7 et Q8** : pays multiples, d'où un filtre par expression régulière (le filtre exact donne 0 en Q7) ; « depuis 2020 » est lu comme 2020 inclus.
- **Q9, Q11, Q13 et Q14** : valeurs brutes sans nettoyage (espace finale, variante mal encodée, « Missing »), moyenne simple sur les 199 pays arrondie à deux décimales.
- **Q3** : `findOne()` a renvoyé la série `s2` et non `s1`, car l'ordre naturel n'est pas garanti ; seule la liste des champs est la réponse.

CONCLUSION

Cette corbeille a permis de manipuler MongoDB en ligne de commande, de l'installation aux seize requêtes :

1. **l'import** : `mongoimport` charge un CSV ou un JSON dans une collection en une commande, sans schéma préalable. Les 34 363 documents sont arrivés sans échec ;
2. **les requêtes simples** : `countDocuments`, `distinct` et `find` avec filtre, tri et limite répondent à la plupart des questions ;
3. **l'agrégation** : `$group`, `$split`, `$unwind` et `$trim` sont nécessaires dès que la donnée n'est pas dans la forme attendue, comme les pays multiples de Q12.

Le point à retenir est que l'absence de schéma impose de bien connaître les données avant de les interroger : un champ multi-valeurs, une durée en texte, des cases vides importées comme des chaînes ou une valeur « Missing » donnent des réponses fausses si l'on ne les anticipe pas. Par rapport à Hive, MongoDB se prête mieux aux documents imbriqués comme les restaurants et se charge plus simplement ; Hive reste plus proche du SQL pour des jointures et des agrégats sur des tables régulières.